Snake Arcade

Informe del Proyecto de Laboratorio de Programación III

Por Najmah Karyme Zablah
22441365

Tabla de contenido

Descripción del Proyecto 3

Bitácora de Investigación 4

SFML 2.6.2 — Simple and Fast Multimedia Library 4

Manejo de Archivos con fstream 5

Algoritmo de Ordenamiento Burbuja (Bubble Sort) 5

Uso de Punteros para la Cabeza de la Serpiente 6

Arquitectura por Pantallas (Screen Architecture) 6

Sistema de Máquina de Estados (State Machine) 7

Generación de Números Aleatorios 7

Efectos Visuales con sf::CircleShape 7

Mejoras Visuales en la Serpiente 8

Diagramas de Lógica 9

Diagrama de Coordenadas y Movimiento 9

Lógica de Colisiones 9

Referencias 10

Descripción del Proyecto

El presente proyecto corresponde al desarrollo de un videojuego de tipo Snake Arcade implementado en el lenguaje de programación C++ bajo el paradigma de la Programación Orientada a Objetos (POO), utilizando la librería gráfica SFML 2.6.2 (Simple and Fast Multimedia Library). El objetivo principal del proyecto es desarrollar un motor de juego basado en el manejo de una lista de coordenadas que representa la posición de la serpiente en un tablero bidimensional, implementando los conceptos fundamentales estudiados en el curso: estructuras (struct), clases (class), punteros, manejo de archivos con fstream y algoritmos de ordenamiento.

El juego contempla un sistema de tres niveles de dificultad progresiva. El primer nivel presenta velocidad constante sin obstáculos internos, permitiendo además que la serpiente atraviese los muros y reaparezca en el extremo opuesto del tablero (wrap-around). El segundo nivel introduce muros perimetrales letales y una velocidad incremental conforme el jugador acumula puntos. El tercer nivel incorpora obstáculos internos generados de forma aleatoria al inicio de cada partida, así como aceitunas doradas que alteran temporalmente la velocidad del juego. Esta progresión garantiza una experiencia de usuario completa, desafiante y entretenida.

Adicionalmente, el proyecto incorpora un sistema de cuentas de usuario con funciones de registro, inicio de sesión y eliminación de cuenta, cuyos datos se almacenan de forma persistente en archivos de texto mediante fstream. También se implementó un sistema de puntuaciones altas (high scores) con ordenamiento por el algoritmo de burbuja (bubble sort), presentado al jugador a través de una pantalla dedicada con pestañas (tabs) por nivel.

Bitácora de Investigación

SFML 2.6.2 — Simple and Fast Multimedia Library

SFML es una librería multimedia multiplataforma de código abierto escrita en C++ que proporciona acceso sencillo a los componentes básicos de un sistema multimedia: ventanas, gráficos 2D, audio y red. Para este proyecto se utilizaron exclusivamente el módulo gráfico (sfml-graphics), el módulo de ventana (sfml-window) y el módulo del sistema (sfml-system). La versión 2.6.2 fue elegida por ser la versión estable más reciente de la rama 2.x y por contar con amplia documentación y compatibilidad con el compilador MinGW 64-bit en el entorno de Qt Creator.

El módulo sfml-graphics provee las clases fundamentales utilizadas a lo largo del proyecto: sf::RenderWindow como superficie de dibujo principal, sf::RectangleShape para dibujar los segmentos de la serpiente, obstáculos y paneles de interfaz, sf::Sprite y sf::Texture para la representación de imágenes PNG, sf::Text y sf::Font para el renderizado de texto, y sf::CircleShape para los efectos de resplandor (glow) alrededor de los coleccionables. La clase sf::Clock fue empleada para controlar el ciclo de actualización del juego, separando el tiempo de movimiento de la serpiente (controlado por la velocidad del nivel) del tiempo de animaciones y overlays visuales.

El sistema de eventos de SFML fue investigado para lograr la captura de entradas de teclado y ratón de forma no bloqueante. A través de ventana.pollEvent(evento) se procesan los eventos acumulados en cada fotograma sin detener la ejecución del hilo principal, lo que cumple con el requisito académico de lógica de teclado no bloqueante. Se investigó la diferencia entre sf::Event::KeyPressed (evento puntual al presionar una tecla) y sf::Keyboard::isKeyPressed() (estado continuo de la tecla), optando por el primero para la dirección de la serpiente con el fin de evitar cambios de dirección no intencionales por repetición automática del sistema operativo.

Manejo de Archivos con fstream

La clase fstream de la biblioteca estándar de C++ (cabecera <fstream>) fue investigada y aplicada para la persistencia de dos tipos de datos: los registros de usuarios y las puntuaciones altas. Se utilizaron los modos std::ofstream para escritura (sobrescritura completa del archivo) y std::ifstream para lectura línea a línea mediante std::getline(). El formato elegido para la serialización de datos es texto plano delimitado por el carácter '|', lo que permite una lectura robusta mediante la función std::string::find() para localizar el delimitador y std::string::substr() para extraer cada campo. Este enfoque evita dependencias externas de serialización y es completamente legible por humanos, facilitando la depuración durante el desarrollo.

Para el archivo de usuarios (usuarios.txt) cada línea almacena el nombre de usuario y la contraseña separados por '|'. Para el archivo de puntuaciones (highscores.txt) cada línea almacena el nombre del jugador, el puntaje numérico y el nivel, también delimitados por '|'. La función stoi() fue utilizada para convertir los campos numéricos leídos como cadenas de texto a enteros. Se investigó la importancia de verificar que los delimitadores existan antes de extraer subcadenas, implementando comprobaciones con std::string::npos para evitar excepciones en tiempo de ejecución ante archivos malformados o vacíos.

Algoritmo de Ordenamiento Burbuja (Bubble Sort)

El algoritmo de ordenamiento por burbuja fue implementado en el método Juego::ordenarPuntajes() para ordenar el vector de registros de tipo PuntajeRecord de mayor a menor puntaje. El algoritmo realiza iteraciones anidadas comparando pares de elementos adyacentes e intercambiándolos si el elemento de la izquierda es menor que el de la derecha, garantizando que los valores más altos "burbujeeen" hacia el inicio del vector. Se investigó la complejidad temporal del algoritmo ($O(n^2)$ en el peor caso) y se determinó que es completamente adecuada para el contexto del proyecto, dado que el número máximo de registros almacenados es diez. El resultado de este ordenamiento se aprovecha directamente en la pantalla de high scores para presentar siempre al jugador los mejores diez resultados por nivel.

Uso de Punteros para la Cabeza de la Serpiente

El uso de punteros fue investigado y aplicado de forma específica en el manejo de la posición de la cabeza de la serpiente, tal como lo requieren los criterios de evaluación del proyecto. El método Serpiente::getCabeza() retorna un puntero de tipo Coordenada* apuntando al primer elemento del vector interno std::vector<Coordenada>, es decir, &cuerpo[0]. Este puntero es utilizado directamente en Juego::verificarColisiones() para leer y modificar la posición de la cabeza sin copiar la estructura. Esto permite, por ejemplo, aplicar el comportamiento de wrap-around en el Nivel 1 modificando los valores de cabeza->x y cabeza->y en su ubicación original dentro del vector, sin necesidad de redibujar toda la estructura de datos ni realizar asignaciones adicionales.

Se investigó la diferencia entre retornar un puntero (Coordenada*) y retornar una referencia (Coordenada&) para este caso de uso. Se optó por el puntero para cumplir explícitamente con el criterio académico de uso de punteros, y porque la semántica de "dirección de memoria modificable" es más clara en este contexto que una referencia.

Arquitectura por Pantallas (Screen Architecture)

Se investigó el patrón de diseño de arquitectura por pantallas (screen-based architecture) como alternativa a un renderizador monolítico. En lugar de concentrar toda la lógica de dibujo y manejo de eventos en una única clase Renderer gigante, el proyecto utiliza clases independientes: PantallaMenu para las pantallas de login y registro, PantallaSeleccion para el menú principal y selección de niveles, PantallaJuego para el tablero de juego activo con sus overlays, y PantallaScores para la tabla de puntuaciones altas. Cada pantalla expone únicamente dos métodos públicos: dibujar() y procesarEventos(), y la clase Renderer actúa como coordinador que delega al objeto correcto según el estado actual del juego (EstadoJuego). Este patrón mejora significativamente la cohesión del código, reduce el acoplamiento entre componentes y facilita el mantenimiento y la extensión futura del proyecto.

Sistema de Máquina de Estados (State Machine)

Para controlar el flujo entre las diferentes pantallas y estados del juego se implementó una máquina de estados simple basada en un enum class llamado EstadoJuego, con los valores MENU, CREAR_CUENTA, SELECCION, NIVELES, CONFIRMAR_ELIMINAR, JUGANDO, PAUSANDO, GAME_OVER e HIGH_SCORES. Se investigó el uso de enum class en C++11 y versiones posteriores como alternativa a los enum clásicos, ya que proporciona un espacio de nombres propio (evitando colisiones de nombres), tipado fuerte (el compilador rechaza comparaciones accidentales con enteros) y mayor claridad semántica. El mismo patrón se aplicó para representar los niveles de dificultad mediante enum class Nivel con los valores UNO, DOS y TRES.

Generación de Números Aleatorios

Para la generación de posiciones aleatorias de aceitunas, estampillas y obstáculos se investigó el uso de las funciones rand() y srand() de la cabecera <cstdlib>, junto con time(nullptr) de <ctime> para inicializar la semilla del generador en función del tiempo actual del sistema. Esto garantiza que cada partida genere una distribución diferente de elementos. Se implementó además una función auxiliar posicionLibre() que verifica que la nueva posición generada no coincida con ningún segmento del cuerpo de la serpiente, ningún obstáculo ya existente ni los coleccionables activos, evitando superposiciones visuales y lógicas.

Efectos Visuales con sf::CircleShape

Para mejorar la experiencia visual se investigó la técnica de glow por capas usando sf::CircleShape con transparencia alfa decreciente. El método PantallaJuego::dibujarGlow() dibuja múltiples círculos concéntricos con el mismo color pero con valores de alfa progresivamente menores hacia afuera, simulando un halo de luz alrededor de los coleccionables. Este efecto se aplica sobre las aceitunas (con glow verde o dorado según el tipo) y sobre las estampillas (con glow blanco). Se utilizó sf::Color con el cuarto componente de tipo sf::Uint8 para controlar la

transparencia de cada capa. El número de capas y el radio son parámetros configurables, lo que permite ajustar el efecto según el tipo de elemento.

Mejoras Visuales en la Serpiente

Durante el desarrollo se realizaron mejoras adicionales a la representación visual de la cabeza de la serpiente dentro de PantallaJuego::dibujarTablero(). Se investigaron y aplicaron las siguientes técnicas de dibujo vectorial con SFML:

Ojos dinámicos: se añadieron dos ojos a la cabeza de la serpiente que se reposicionan automáticamente según la dirección actual de movimiento. La dirección se deduce en tiempo real comparando las coordenadas de cuerpo[0] y cuerpo[1], normalizando el resultado para manejar correctamente el wrap-around del Nivel 1 (donde un salto de 29 celdas indica movimiento en la dirección opuesta). Cada ojo se compone de tres capas: un círculo blanco (esclerótica), una pupila negra y un pequeño reflejo blanco semitransparente, todos renderizados con sf::CircleShape.

Sombrero de copa: se dibujó un sombrero negro sobre la celda de la cabeza usando tres primitivas sf::RectangleShape: el ala del sombrero, el cuerpo principal y una cinta decorativa. La cinta utiliza el color morado de la paleta del juego (RGB 75, 55, 110) para mantener coherencia visual con el resto de la interfaz. Una cuarta forma con alpha reducido simula una sombra suave debajo del ala, añadiendo profundidad al conjunto.

Tamaño de sprites: los sprites PNG de aceitunas y cajas de obstáculos fueron escalados de 36 a 44 píxeles para mejorar la visibilidad dentro del tablero de 20 px por celda, manteniendo el comportamiento de colisión original basado en coordenadas de celda enteras.

Diagramas de Lógica

Diagrama de Coordenadas y Movimiento

La serpiente se representa internamente como un std::vector<Coordenada> donde cada Coordenada contiene un par de enteros (x, y) que referencian una celda del tablero. El tablero tiene dimensiones de 30 celdas de ancho por 28 celdas de alto, y cada celda ocupa 20 píxeles en pantalla. El índice 0 del vector corresponde siempre a la cabeza de la serpiente. En cada ciclo de actualización, el método mover() calcula la nueva posición de la cabeza sumando la dirección actual (direccionX, direccionY) a las coordenadas de la cabeza, inserta esta nueva coordenada al inicio del vector con insert(cuerpo.begin(), nuevaCabeza) y, si la serpiente no está en modo de crecimiento, elimina el último elemento con pop_back(). Cuando la serpiente come una aceituna, se activa la bandera creciendo = true, lo que hace que en el siguiente ciclo de movimiento el pop_back() sea omitido, efectivamente añadiendo un segmento al cuerpo.

Lógica de Colisiones

El sistema de colisiones se ejecuta en cada ciclo de actualización dentro del método Juego::verificarColisiones(). En primer lugar se obtiene un puntero a la cabeza mediante getCabeza(). Para el Nivel 1 se aplica aritmética modular a las coordenadas de la cabeza para implementar el wrap-around. Para los Niveles 2 y 3 se verifica si las coordenadas de la cabeza están fuera de los límites del tablero, en cuyo caso el estado cambia a GAME_OVER. Posteriormente se recorre el vector del cuerpo desde el índice 1 para detectar si la cabeza coincide con algún segmento del propio cuerpo (auto-colisión). Para el Nivel 3 se recorre adicionalmente el vector de obstáculos. La colisión con la aceituna se detecta comparando la coordenada de la cabeza con la posición de la aceituna mediante el operador == sobrecargado en struct Coordenada. La colisión con la estampilla utiliza un radio de tolerancia de una celda en ambas dimensiones para compensar el mayor tamaño visual del sprite.

Referencias

- SFML Development Team. (2024). SFML 2.6.2 — Official Documentation. https://www.sfml-dev.org/documentation/2.6.2/
- SFML Development Team. (2023). SFML Tutorials — Getting Started with SFML. https://www.sfml-dev.org/tutorials/2.6/
- cppreference.com. (2024). std::fstream — C++ Reference. https://en.cppreference.com/w/cpp/io/basic_fstream
- cppreference.com. (2024). std::vector — C++ Reference. https://en.cppreference.com/w/cpp/container/vector
- cppreference.com. (2024). enum declaration — C++ Reference. https://en.cppreference.com/w/cpp/language/enum
- cppreference.com. (2024). std::string::find — C++ Reference. https://en.cppreference.com/w/cpp/string/basic_string/find
- GeeksforGeeks. (2023). Bubble Sort Algorithm. https://www.geeksforgeeks.org/bubble-sort-algorithm/
- Qt Company. (2024). Qt Creator Manual. https://doc.qt.io/qtcreator/
- SFML Development Team. (2024). sf::RenderWindow Documentation. https://www.sfml-dev.org/documentation/2.6.2/classsf_1_1RenderWindow.php
- SFML Development Team. (2024). sf::Sprite Documentation. https://www.sfml-dev.org/documentation/2.6.2/classsf_1_1Sprite.php
- SFML Development Team. (2024). sf::Clock Documentation. https://www.sfml-dev.org/documentation/2.6.2/classsf_1_1Clock.php

- cppreference.com. (2024). std::rand — C++ Reference. https://en.cppreference.com/w/cpp/numeric/random/rand

[i]

[ii]

[iii]

[i] @saino_m (2025), “Debonheir” – Estampilla del Mono: https://x.com/saino_m
[ii] @pompokouu (2025), “Doremin” – Estampilla del Erizo: https://x.com/pompokouu
[iii] @nan_tetra (2025), “Seinyan” – Estampilla del Gato: https://x.com/nana_tetra